

Domain 4: Security and Compliance

AWS Certified SysOps Administrator – Associate (SOA-C03)

Exam Weight: ~16% of scored content

Source: AWS Certified SysOps Administrator – Associate Exam Guide

Overview

Domain 4 covers the skills required to implement and manage security controls, enforce compliance policies, and protect data and infrastructure on AWS. A CloudOps engineer must understand how to configure IAM correctly, audit access, manage multi-account environments securely, protect data at rest and in transit, store secrets safely, and act on findings from AWS security services.

Security on AWS is a shared responsibility: AWS secures the underlying infrastructure, while you are responsible for securing what you build on top of it — including identity, access, data, and network controls.

Task 4.1: Implement and Manage Security and Compliance Tools and Policies

Skill 4.1.1 – Implement AWS IAM Features

What is AWS IAM?

AWS Identity and Access Management (IAM) is the service that controls who can authenticate (sign in) and what they are authorized to do (permissions) in your AWS account. IAM is global — it is not Region-specific.

Source: IAM User Guide – AWS Identity and Access Management

Password Policies

An IAM **account password policy** enforces complexity and rotation requirements for IAM user passwords. You configure it at the account level and it applies to all IAM users.

Key settings:

- Minimum password length (recommended: 14+ characters)

- Require uppercase, lowercase, numbers, and non-alphanumeric characters
- Password expiration (e.g., every 90 days)
- Prevent password reuse (e.g., remember last 24 passwords)
- Allow users to change their own passwords

```
# Set a password policy via AWS CLI
aws iam update-account-password-policy \
  --minimum-password-length 14 \
  --require-uppercase-characters \
  --require-lowercase-characters \
  --require-numbers \
  --require-symbols \
  --max-password-age 90 \
  --password-reuse-prevention 24
```

Source: Setting an account password policy – AWS IAM

Multi-Factor Authentication (MFA)

MFA adds a second layer of authentication beyond a password. AWS supports several MFA device types:

MFA Type	Description
Passkey / Security key	FIDO2-compliant hardware key (e.g., YubiKey); phishing-resistant
Virtual MFA app	TOTP-based app (e.g., Google Authenticator, Authy); generates 6-digit codes
Hardware TOTP token	Physical device that generates time-based one-time passwords

MFA recommendations:

- Require MFA for the root user — always.
- Require MFA for all IAM users with console access.
- Use IAM condition keys (`aws:MultiFactorAuthPresent`) to enforce MFA for sensitive API calls.
- For human users, prefer federation with an identity provider (IdP) that enforces MFA at the IdP level.

```
// IAM policy condition to require MFA
{
  "Condition": {
    "BoolIfExists": {
      "aws:MultiFactorAuthPresent": "true"
    }
  }
}
```

IAM Roles

An **IAM role** is an identity with a set of permissions that can be assumed by trusted entities — AWS services, IAM users, federated users, or other AWS accounts. Roles issue **temporary security credentials** via AWS STS, which is more secure than long-term access keys.

Common role use cases:

Use Case	Description
EC2 instance profile	Attach a role to an EC2 instance so applications can call AWS APIs without embedding credentials
Cross-account access	Allow a principal in Account A to assume a role in Account B
Service role	Allow an AWS service (e.g., Lambda, CodeDeploy) to act on your behalf
Break-glass role	An emergency role with elevated permissions, used only in incident response

Trust policy example (allows EC2 to assume the role):

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "ec2.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
```

Federated Identity

Federation allows users authenticated by an external identity provider (IdP) to access AWS without creating individual IAM users. AWS supports two federation standards:

- **SAML 2.0** – Integrates with enterprise IdPs like Microsoft Active Directory Federation Services (ADFS) or Okta. Users authenticate with their corporate credentials and receive temporary AWS credentials.
- **OpenID Connect (OIDC)** – Used for web and mobile applications. Supports IdPs like Google, Facebook, or any OIDC-compliant provider.

AWS IAM Identity Center (formerly AWS SSO) is the recommended service for workforce federation. It provides a single sign-on portal, integrates with AWS Organizations, and supports attribute-based access control (ABAC).

Source: IAM Identity Center – AWS IAM Identity Center

Resource-Based Policies

While identity-based policies are attached to IAM principals, **resource-based policies** are attached directly to AWS resources (e.g., S3 buckets, KMS keys, SQS queues, Lambda functions). They specify who can access the resource and what actions they can perform.

Key points:

- Resource-based policies can grant cross-account access without requiring the external principal to assume a role.
- They must explicitly allow the action; an implicit deny in a resource policy blocks access even if the identity policy allows it.
- The `Principal` element is required in resource-based policies (unlike identity-based policies).

Policy Conditions

IAM **policy conditions** let you restrict when a policy takes effect. Common condition keys:

Condition Key	Example Use
<code>aws:SourceIp</code>	Restrict access to specific IP ranges
<code>aws:RequestedRegion</code>	Limit actions to specific AWS Regions
<code>aws:MultiFactorAuthPresent</code>	Require MFA for sensitive operations
<code>aws:PrincipalTag</code>	ABAC — match tags on the principal
<code>s3:prefix</code>	Restrict S3 access to specific key prefixes
<code>kms:ViaService</code>	Allow KMS key use only from a specific AWS service

Source: Policies and permissions in IAM

IAM Security Best Practices

Source: Security best practices in IAM

- **Use federation** for human users instead of creating IAM users with long-term credentials.
- **Use IAM roles** for workloads (EC2, Lambda, ECS tasks) instead of embedding access keys.

- **Apply least-privilege permissions** — start with AWS managed policies, then refine using IAM Access Analyzer.
- **Rotate access keys** regularly if long-term credentials are unavoidable.
- **Enable MFA** for all users with console access.
- **Use permissions boundaries** to delegate permissions management safely.
- **Use IAM Access Analyzer** to identify unused permissions and external access.

Skill 4.1.2 – Troubleshoot and Audit Access Issues

AWS CloudTrail

AWS CloudTrail records API calls made in your AWS account, providing a complete audit trail of who did what, when, and from where. Every management event (console action, CLI call, SDK call) is logged.

Source: What is AWS CloudTrail?

Key CloudTrail concepts:

Concept	Description
Trail	Configuration that delivers log files to an S3 bucket; can cover one Region or all Regions
Event	A record of an API call; includes <code>eventName</code> , <code>userIdentity</code> , <code>sourceIPAddress</code> , <code>requestParameters</code> , <code>responseElements</code>
Management events	Control-plane operations (e.g., <code>CreateBucket</code> , <code>RunInstances</code> , <code>AssumeRole</code>) — logged by default
Data events	Data-plane operations (e.g., S3 <code>GetObject</code> , Lambda <code>Invoke</code>) — must be explicitly enabled
Insights events	Detects unusual API activity patterns (e.g., a spike in <code>TerminateInstances</code> calls)
CloudTrail Lake	Managed data lake for querying CloudTrail events using SQL

Troubleshooting access denials with CloudTrail:

1. Search CloudTrail for the `AccessDenied` or `UnauthorizedOperation` error.
2. Note the `userIdentity` (who made the call), `eventName` (what action), and `errorMessage`.
3. Use the IAM policy simulator to test whether the principal's policies allow the action.
4. Check for SCPs, permission boundaries, or resource-based policies that may be blocking access.

```
# Search CloudTrail for access denied events in the last 24 hours
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=AccessDenied \
  --start-time $(date -d '24 hours ago' --iso-8601=seconds)
```

IAM Access Analyzer

IAM Access Analyzer helps you identify resources in your account (or organization) that are shared with external entities, detect unused permissions, and validate policies against AWS best practices.

Source: Using AWS IAM Access Analyzer

Key capabilities:

Capability	Description
External access analysis	Identifies S3 buckets, KMS keys, IAM roles, Lambda functions, SQS queues, and Secrets Manager secrets accessible from outside your account or organization
Unused access analysis	Identifies IAM roles and users with unused permissions, access keys, and passwords
Policy validation	Checks policies for syntax errors, security warnings, and deviations from best practices
Policy generation	Analyzes CloudTrail activity to generate a least-privilege policy for a principal
Custom policy checks	Validates that a new policy does not grant more access than a reference policy

Analyzer scope:

- **Account analyzer** – Finds resources accessible from outside the account.
- **Organization analyzer** – Finds resources accessible from outside the organization (requires AWS Organizations).

Workflow for remediating findings:

1. Review the finding in the IAM Access Analyzer console.
2. Determine whether the external access is intentional (archive the finding) or unintentional (remediate).
3. Update the resource policy or IAM role trust policy to remove unintended access.
4. Re-analyze to confirm the finding is resolved.

IAM Policy Simulator

The **IAM policy simulator** lets you test and troubleshoot identity-based policies, permissions boundaries, SCPs, and resource-based policies without making actual API calls.

Source: IAM policy testing with the IAM policy simulator

How it works:

1. Select a user, group, or role (or paste a policy directly).
2. Choose the AWS service and action to test (e.g., `s3:GetObject`).
3. Optionally specify resource ARNs and condition context keys.

4. Run the simulation — the result shows **Allowed** or **Denied**, and which policy statement caused the decision.

Use cases:

- Verify that a new IAM policy grants the intended permissions before attaching it.
- Diagnose why a user is getting `AccessDenied` errors.
- Test the effect of SCPs on member account principals.
- Validate permissions boundaries.

Skill 4.1.3 – Implement Multi-Account Strategies Securely

Managing multiple AWS accounts is a best practice for isolating workloads, enforcing security boundaries, and simplifying billing. AWS provides several services to manage this at scale.

AWS Organizations

AWS Organizations lets you centrally manage multiple AWS accounts under a single management account. You can group accounts into **Organizational Units (OUs)** and apply policies at the organization, OU, or account level.

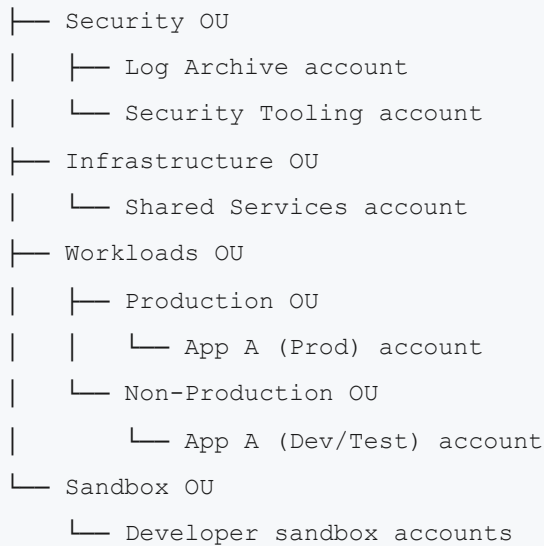
Source: What is AWS Organizations?

Key concepts:

Concept	Description
Management account	The root account that creates and manages the organization; cannot be a member account
Member account	Any account that belongs to the organization
Organizational Unit (OU)	A logical grouping of accounts within the organization hierarchy
Service Control Policy (SCP)	A policy that sets the maximum permissions for principals in member accounts
Delegated administrator	A member account granted administrative permissions for a specific AWS service (e.g., Security Hub, GuardDuty)

Recommended OU structure:

Root



Source: AWS multi-account strategy – AWS Control Tower

Service Control Policies (SCPs)

SCPs are a type of Organizations policy that set the maximum permissions available to IAM principals in member accounts. They do not grant permissions — they restrict what permissions can be granted.

Source: Service control policies (SCPs) – AWS Organizations

SCP evaluation logic:

- An action is allowed only if it is **not denied by any SCP** in the hierarchy AND **explicitly allowed by an IAM policy**.
- SCPs apply to all principals in the account **except the management account** and the root user of the management account.
- SCPs do not affect service-linked roles.

Common SCP use cases:


```
// Deny actions outside approved Regions
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {
      "aws:RequestedRegion": ["us-east-1", "eu-west-1"]
    }
  }
}
```

```
// Prevent disabling CloudTrail
{
  "Effect": "Deny",
  "Action": [
    "cloudtrail:StopLogging",
    "cloudtrail:DeleteTrail"
  ],
  "Resource": "*"
}
```

AWS Control Tower

AWS Control Tower provides a pre-configured, opinionated landing zone for multi-account environments. It automates the setup of AWS Organizations, account vending, guardrails (preventive and detective controls), and a centralized audit log.

Key components:

- **Landing zone** – A well-architected multi-account environment set up by Control Tower.
- **Guardrails** – Pre-packaged governance rules implemented as SCPs (preventive) or AWS Config rules (detective).
- **Account Factory** – A self-service portal for provisioning new accounts with pre-approved configurations.
- **Control Tower Dashboard** – Provides compliance status across all enrolled accounts.

AWS IAM Identity Center (Workforce SSO)

For multi-account environments, **IAM Identity Center** is the recommended way to manage human access. It provides:

- A single sign-on portal for all AWS accounts in the organization.
- Permission sets that define what a user can do in a given account.
- Integration with external IdPs (Active Directory, Okta, Azure AD).
- Attribute-based access control (ABAC) using user attributes from the IdP.

Source: What is IAM Identity Center?

Skill 4.1.4 – Implement Remediation Based on AWS Trusted Advisor Security Checks

What is AWS Trusted Advisor?

AWS Trusted Advisor is an online tool that inspects your AWS environment and provides real-time recommendations across five categories: cost optimization, performance, security, fault tolerance, and service limits. The security checks are particularly relevant for CloudOps engineers.

Source: AWS Trusted Advisor – AWS Support

Key Security Checks

Check	What It Detects	Remediation
MFA on Root Account	Root user does not have MFA enabled	Enable a hardware or virtual MFA device on the root account
IAM Use	No IAM users or roles are being used (root-only access)	Create IAM users/roles; stop using root credentials for daily tasks
Access Keys Not Rotated	IAM access keys older than 90 days	Rotate or delete unused access keys
Exposed Access Keys	Access keys found in public code repositories	Immediately deactivate the key, investigate for unauthorized use, rotate
S3 Bucket Permissions	S3 buckets with public read or write access	Enable S3 Block Public Access; review bucket policies and ACLs
Security Groups – Unrestricted Access	Security groups with <code>0.0.0.0/0</code> on sensitive ports (e.g., 22, 3389, 1433)	Restrict inbound rules to known IP ranges or use Systems Manager Session Manager instead of SSH
CloudTrail Logging	CloudTrail is not enabled in all Regions	Enable a multi-Region trail and ensure log file validation is on
Amazon RDS Security Group Access Risk	RDS instances with overly permissive security groups	Restrict database security groups to application tier only
Amazon EBS Public Snapshots	EBS snapshots set to public	Change snapshot permissions to private
Amazon RDS Public Snapshots	RDS snapshots set to public	Change snapshot permissions to private

Accessing Trusted Advisor Checks

- **AWS Management Console** – Navigate to AWS Support → Trusted Advisor.
- **AWS Support API** – Use `DescribeTrustedAdvisorChecks` and `DescribeTrustedAdvisorCheckResult` to retrieve check results programmatically.
- **AWS Health Dashboard** – Trusted Advisor findings can surface as health events.
- **Amazon EventBridge** – Trusted Advisor publishes events when check statuses change, enabling automated remediation via Lambda or Systems Manager Automation.

Automating Remediation

You can automate responses to Trusted Advisor findings using EventBridge rules:

1. Create an EventBridge rule that matches `aws.trustedadvisor` events with a specific check ID.
2. Target a Lambda function or Systems Manager Automation runbook.
3. The automation remediates the finding (e.g., restricts a security group, rotates an access key).

```
// EventBridge rule pattern for Trusted Advisor security check changes
{
  "source": ["aws.trustedadvisor"],
  "detail-type": ["Trusted Advisor Check Item Refresh Notification"],
  "detail": {
    "status": ["ERROR", "WARN"],
    "check-name": ["Security Groups - Unrestricted Access"]
  }
}
```

Note: Business Support or Enterprise Support is required to access all Trusted Advisor checks via the API. Developer Support and free tier accounts have access to a limited set of checks.

Skill 4.1.5 – Enforce Compliance Requirements

Region and Service Restrictions

Compliance frameworks (e.g., GDPR, FedRAMP, HIPAA) often require that data remain within specific geographic boundaries or that only approved services are used. AWS provides several mechanisms to enforce these requirements.

Using SCPs to restrict Regions:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyNonApprovedRegions",
    "Effect": "Deny",
    "NotAction": [
      "iam:*",
      "organizations:*",
      "support:*",
      "budgets:*",
      "cloudfront:*",
      "route53:*",
      "waf:*"
    ],
    "Resource": "*",
    "Condition": {
      "StringNotEquals": {
        "aws:RequestedRegion": ["us-east-1", "us-west-2"]
      }
    }
  }]
}
```

Note: Global services (IAM, Route 53, CloudFront, etc.) are excluded from the Region restriction using `NotAction` because they do not operate in a specific Region.

Using SCPs to restrict services:

```
{
  "Effect": "Deny",
  "Action": [
    "ec2:RunInstances"
  ],
  "Resource": "arn:aws:ec2:*:*:instance/*",
  "Condition": {
    "StringNotEquals": {
      "ec2:InstanceType": ["t3.micro", "t3.small", "t3.medium"]
    }
  }
}
```

AWS Config for Compliance

AWS Config continuously records the configuration of AWS resources and evaluates them against desired-state rules. It provides a compliance dashboard showing which resources are compliant or non-compliant.

Source: What is AWS Config?

Key AWS Config concepts:

Concept	Description
Configuration recorder	Records resource configuration changes and stores them as configuration items
Config rule	Evaluates resource configurations against a desired state; can be AWS managed or custom (Lambda-backed)
Conformance pack	A collection of Config rules and remediation actions deployed as a single unit
Remediation action	An SSM Automation document triggered when a resource is non-compliant
Aggregator	Collects Config data from multiple accounts and Regions into a single view

Common managed Config rules for compliance:

Rule	What It Checks
cloudtrail-enabled	CloudTrail is enabled in the account
mfa-enabled-for-iam-console-access	IAM users with console access have MFA enabled
s3-bucket-public-read-prohibited	S3 buckets do not allow public read access
encrypted-volumes	EBS volumes are encrypted
rds-storage-encrypted	RDS DB instances have storage encryption enabled
iam-password-policy	Account password policy meets specified requirements
restricted-ssh	Security groups do not allow unrestricted SSH (port 22)

Conformance packs provide pre-built collections of rules aligned to compliance frameworks:

- Operational-Best-Practices-for-PCI-DSS
 - Operational-Best-Practices-for-HIPAA-Security
 - Operational-Best-Practices-for-CIS-AWS-Foundations-Benchmark
-

Task 4.2: Implement Strategies to Protect Data and Infrastructure

Skill 4.2.1 – Implement and Enforce a Data Classification Scheme

Why Data Classification Matters

Data classification is the process of categorizing data based on its sensitivity and the impact of unauthorized disclosure, modification, or loss. It is the foundation of a data protection strategy — you cannot protect data appropriately if you do not know what you have and how sensitive it is.

Source: Data classification overview – AWS Whitepaper

Classification Levels (Example)

Level	Description	Examples
Public	No harm if disclosed	Marketing materials, public documentation
Internal	Low sensitivity; for internal use	Internal wikis, non-sensitive operational data
Confidential	Moderate sensitivity; limited distribution	Business plans, employee data, financial reports
Restricted	High sensitivity; strict controls required	PII, PHI, payment card data, credentials

Implementing Classification on AWS

Tagging is the primary mechanism for enforcing data classification on AWS resources:

```
# Tag an S3 bucket with its data classification
aws s3api put-bucket-tagging \
  --bucket my-sensitive-bucket \
  --tagging 'TagSet=[{Key=DataClassification,Value=Restricted}]'
```

Use **AWS Config rules** and **SCPs** to enforce controls based on classification tags:

- Require encryption for resources tagged `DataClassification=Restricted`.
- Deny public access to resources with sensitive classification tags.
- Require specific KMS keys for resources tagged as restricted.

Amazon Macie

Amazon Macie is a fully managed data security service that uses machine learning to automatically discover, classify, and protect sensitive data stored in Amazon S3.

Source: What is Amazon Macie?

Key Macie capabilities:

Capability	Description
Automated sensitive data discovery	Continuously samples S3 objects to identify buckets containing sensitive data
Sensitive data findings	Generates findings when PII, financial data, credentials, or other sensitive data types are detected
Managed data identifiers	Pre-built patterns for common sensitive data types (credit card numbers, SSNs, passport numbers, etc.)
Custom data identifiers	Regex-based patterns for organization-specific sensitive data
Security findings	Identifies S3 buckets with public access, unencrypted storage, or cross-account access

Macie integration with Security Hub: Macie findings are automatically published to AWS Security Hub, providing a centralized view of data security posture alongside findings from GuardDuty, Inspector, and Config.

Skill 4.2.2 – Implement, Configure, and Troubleshoot Encryption at Rest (AWS KMS)

What is AWS KMS?

AWS Key Management Service (AWS KMS) is a managed service that makes it easy to create and control the cryptographic keys used to encrypt your data. KMS keys are protected by FIPS 140-3 validated hardware security modules (HSMs).

Source: AWS Key Management Service

KMS Key Types

Key Type	Description	Use Case
AWS managed key	Created and managed by AWS on your behalf; one per service per Region	Default encryption for services like S3, EBS, RDS
Customer managed key (CMK)	Created and managed by you; full control over key policy, rotation, and deletion	Compliance requirements, cross-account encryption, custom key policies
AWS owned key	Owned and managed entirely by AWS; not visible in your account	Used by some services internally; no customer control

Key Policies

Every KMS key has a **key policy** — a resource-based policy that controls who can use and manage the key. Unlike IAM policies, the key policy is the primary access control mechanism for KMS keys.

```
// Key policy statement allowing an IAM role to use the key
{
  "Sid": "Allow use of the key",
  "Effect": "Allow",
  "Principal": {
    "AWS": "arn:aws:iam::123456789012:role/MyAppRole"
  },
  "Action": [
    "kms:Encrypt",
    "kms:Decrypt",
    "kms:ReEncrypt*",
    "kms:GenerateDataKey*",
    "kms:DescribeKey"
  ],
  "Resource": "*"
}
```

Envelope Encryption

AWS services use **envelope encryption** when encrypting data with KMS:

1. KMS generates a **data key** (a symmetric key used to encrypt the actual data).
2. The data key is used to encrypt the data locally (fast, no size limit).
3. The data key itself is encrypted by the KMS key and stored alongside the encrypted data.
4. To decrypt, the encrypted data key is sent to KMS, decrypted, and used to decrypt the data.

This approach means KMS only handles small key operations, not large data volumes.

Key Rotation

- **Automatic rotation** – For customer managed keys, you can enable automatic rotation (every year by default, or on a custom schedule). AWS generates new key material but keeps the old material to decrypt data encrypted with previous versions.
- **On-demand rotation** – Manually trigger rotation at any time.
- **Manual rotation** – Create a new KMS key and update all references (aliases, service configurations) to point to the new key.

```
# Enable automatic key rotation

aws kms enable-key-rotation --key-id alias/my-key


# Rotate a key on demand

aws kms rotate-key-on-demand --key-id alias/my-key
```

Source: Rotate AWS KMS keys

Encryption at Rest for Common Services

Service	Encryption Mechanism
Amazon S3	SSE-S3 (AES-256, AWS managed), SSE-KMS (customer or AWS managed KMS key), SSE-C (customer-provided key), DSSE-KMS (dual-layer)
Amazon EBS	AES-256 encryption using KMS; enable account-level default encryption to encrypt all new volumes automatically
Amazon RDS	Enable encryption at creation time; uses KMS; encrypted snapshots, read replicas, and automated backups
Amazon DynamoDB	Default encryption with AWS owned key; optionally use AWS managed or customer managed KMS key
AWS Secrets Manager	Secrets encrypted with KMS (AWS managed or customer managed key)
Amazon SQS	Server-side encryption using KMS

Troubleshooting KMS Issues

Issue	Cause	Resolution
<code>AccessDeniedException</code> on <code>kms:Decrypt</code>	IAM role lacks permission in key policy or IAM policy	Add the role to the key policy; check IAM policy for <code>kms:Decrypt</code>
<code>DisabledException</code>	The KMS key is disabled	Re-enable the key in the KMS console
<code>InvalidKeyUsageException</code>	Using an asymmetric key for symmetric operations	Use the correct key type for the operation
<code>KMSInvalidStateException</code>	Key is pending deletion	Cancel the deletion or use a different key
Cross-account decryption fails	Target account principal not in key policy	Add the external principal to the key policy

Skill 4.2.3 – Implement, Configure, and Troubleshoot Encryption in Transit (AWS Certificate Manager)

Why Encryption in Transit Matters

Encryption in transit protects data as it moves between clients and servers, or between services, by ensuring that even if traffic is intercepted, it cannot be read. The standard protocol is **TLS (Transport Layer Security)**, which supersedes SSL.

Source: SECO9-BPO2 Enforce encryption in transit – AWS Well-Architected Framework

What is AWS Certificate Manager (ACM)?

AWS Certificate Manager (ACM) is a managed service that provisions, manages, and deploys public and private TLS/SSL certificates for use with AWS services and internal connected resources. ACM handles the complexity of certificate renewal automatically.

Source: What is AWS Certificate Manager?

Certificate Types

Type	Description	Use Case
ACM public certificate	Free, domain-validated certificate issued by Amazon Trust Services	HTTPS for public-facing websites and APIs
ACM private certificate	Issued by AWS Private CA (a private certificate authority you manage)	Internal services, microservices, IoT devices
Imported certificate	Third-party certificate imported into ACM	Certificates from existing CAs; ACM does not auto-renew these

Domain Validation

Before ACM issues a public certificate, it validates that you control the domain. Two validation methods:

- **DNS validation** (recommended) – Add a CNAME record to your DNS configuration. ACM can automatically add this record if the domain is managed in Route 53. Certificates renew automatically as long as the CNAME record exists.
- **Email validation** – ACM sends a validation email to the domain's registered contacts. Requires manual action for each renewal.

ACM Integration with AWS Services

ACM certificates can be deployed to:

- **Elastic Load Balancing (ALB, NLB, CLB)** – Terminate TLS at the load balancer.
- **Amazon CloudFront** – Serve HTTPS content from a CDN; certificate must be in `us-east-1`.
- **Amazon API Gateway** – Secure REST and HTTP APIs.

- **AWS Elastic Beanstalk** – Attach certificates to the environment's load balancer.
- **AWS App Runner, Amazon Cognito** – Managed HTTPS endpoints.

*Note: ACM certificates **cannot** be directly installed on EC2 instances. For EC2, use ACM Private CA to export the certificate, or use a third-party certificate.*

Certificate Renewal

- ACM automatically renews **managed public certificates** 60 days before expiration, provided DNS validation is in place.
- For **imported certificates**, ACM sends expiration notifications via EventBridge and CloudWatch, but renewal is manual.
- Monitor certificate expiration using CloudWatch metric `DaysToExpiry` and set alarms at 45 and 30 days.

```
# List certificates and their expiration dates
aws acm list-certificates --query 'CertificateSummaryList[*].[DomainName,Status]'

# Describe a specific certificate
aws acm describe-certificate --certificate-arn arn:aws:acm:us-east-1:123456789012:certificate/abc123
```

Troubleshooting ACM Issues

Issue	Cause	Resolution
Certificate stuck in <code>PENDING_VALIDATION</code>	DNS CNAME record not added or not propagated	Verify the CNAME record in your DNS provider; allow up to 72 hours for propagation
Certificate not renewing	DNS validation CNAME was removed	Re-add the CNAME record; ACM will retry validation
<code>FAILED</code> status	Domain validation failed (email bounced, DNS misconfigured)	Re-request the certificate; check DNS configuration
CloudFront not using new certificate	Certificate not in <code>us-east-1</code>	ACM certificates for CloudFront must be requested in the <code>us-east-1</code> Region
ALB returning old certificate	Certificate not associated with the HTTPS listener	Update the ALB listener to use the new certificate ARN

Enforcing TLS Across Services

- **ALB** – Configure HTTPS listeners with a security policy that enforces TLS 1.2 or higher (e.g., `ELBSecurityPolicy-TLS13-1-2-2021-06`).
- **CloudFront** – Set the minimum protocol version to TLSv1.2 in the distribution settings.
- **API Gateway** – Use a custom domain with a minimum TLS version of TLS 1.2.

- **S3** – Use a bucket policy to deny requests that do not use HTTPS:

```
{
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:*",
  "Resource": [
    "arn:aws:s3::my-bucket",
    "arn:aws:s3::my-bucket/*"
  ],
  "Condition": {
    "Bool": {
      "aws:SecureTransport": "false"
    }
  }
}
```

Skill 4.2.4 – Securely Store Secrets Using AWS Services

The Problem with Hard-Coded Credentials

Embedding database passwords, API keys, or other secrets directly in application code or configuration files is a critical security risk. If the code is committed to a repository or the instance is compromised, the credentials are exposed. AWS provides two primary services for secure secret storage.

AWS Secrets Manager

AWS Secrets Manager is a fully managed service for storing, retrieving, and automatically rotating secrets such as database credentials, API keys, and OAuth tokens.

Source: What is AWS Secrets Manager?

Key features:

Feature	Description
Automatic rotation	Rotates secrets on a schedule using a Lambda function; native support for RDS, Redshift, DocumentDB, and other services
KMS encryption	All secrets are encrypted at rest using KMS (AWS managed or customer managed key)
Fine-grained access control	IAM policies and resource-based policies control who can retrieve or manage each secret
Versioning	Maintains multiple versions of a secret (e.g., <code>AWSCURRENT</code> , <code>AWSPENDING</code> , <code>AWSPREVIOUS</code>) during rotation
Cross-account access	Resource-based policies allow other accounts to retrieve secrets
Audit trail	All secret access and management operations are logged in CloudTrail

Retrieving a secret in application code:

```
import boto3
import json

client = boto3.client('secretsmanager', region_name='us-east-1')
response = client.get_secret_value(SecretId='prod/myapp/db-credentials')
secret = json.loads(response['SecretString'])
db_password = secret['password']
```

Automatic rotation workflow:

1. Secrets Manager invokes a Lambda rotation function.
2. The function creates a new credential in the target service (e.g., a new RDS password).
3. The new credential is stored as `AWSPENDING` in Secrets Manager.
4. The function tests the new credential.
5. If the test passes, the new credential becomes `AWSCURRENT` and the old one becomes `AWSPREVIOUS`.

```
# Enable automatic rotation (every 30 days)
aws secretsmanager rotate-secret \
  --secret-id prod/myapp/db-credentials \
  --rotation-rules AutomaticallyAfterDays=30
```

AWS Systems Manager Parameter Store

Parameter Store is a capability of AWS Systems Manager that provides secure, hierarchical storage for configuration data and secrets. It is a lighter-weight alternative to Secrets Manager for non-rotating secrets and configuration values.

Source: AWS Systems Manager Parameter Store

Parameter types:

Type	Description
<code>String</code>	Plain text value
<code>StringList</code>	Comma-separated list of plain text values
<code>SecureString</code>	Encrypted value using KMS

Comparison: Secrets Manager vs. Parameter Store

Feature	Secrets Manager	Parameter Store
Automatic rotation	Yes (native + Lambda)	No (manual or custom Lambda)
Cost	Per secret per month + API calls	Free tier available; charges for advanced parameters
Secret size	Up to 65,536 bytes	Up to 8 KB (standard), 8 KB (advanced)
Cross-account access	Yes (resource-based policy)	Limited
CloudFormation integration	<code>{{resolve:secretsmanager:r:...}}</code>	<code>{{resolve:ssm:...}}</code> or <code>{{resolve:ssm-secure:...}}</code>
Best for	Database credentials, API keys requiring rotation	Configuration values, feature flags, non-rotating secrets

Use **Secrets Manager** when you need automatic rotation or cross-account secret sharing. Use **Parameter Store** for configuration data, feature flags, and secrets that do not require rotation.

Skill 4.2.5 – Configure Reports and Remediate Findings from AWS Security Services

AWS provides a suite of security services that continuously monitor your environment and generate findings. A CloudOps engineer must know how to configure these services, interpret their findings, and remediate issues.

AWS Security Hub

AWS Security Hub provides a comprehensive view of your security posture across AWS accounts. It aggregates, normalizes, and prioritizes findings from multiple AWS security services and third-party tools using the **AWS Security Finding Format (ASFF)**.

Source: AWS Security Hub User Guide

Key capabilities:

Capability	Description
Security standards	Pre-built compliance checks (CIS AWS Foundations, PCI DSS, AWS Foundational Security Best Practices, NIST 800-53)
Finding aggregation	Collects findings from GuardDuty, Inspector, Macie, Config, IAM Access Analyzer, and third-party tools
Cross-account aggregation	Designate an administrator account to aggregate findings from all member accounts
Automated response	EventBridge integration for automated remediation via Lambda or Systems Manager
Security Hub policies	Centrally configure Security Hub settings across an AWS Organization

Automated remediation with EventBridge:

```
// EventBridge rule to trigger Lambda when a critical Security Hub finding is created
{
  "source": ["aws.securityhub"],
  "detail-type": ["Security Hub Findings - Imported"],
  "detail": {
    "findings": {
      "Severity": { "Label": ["CRITICAL", "HIGH"] },
      "Workflow": { "Status": ["NEW"] }
    }
  }
}
```

Source: Using EventBridge for automated response and remediation – Security Hub

Amazon GuardDuty

Amazon GuardDuty is a threat detection service that continuously monitors for malicious activity and unauthorized behavior across your AWS accounts. It analyzes CloudTrail logs, VPC Flow Logs, DNS logs, and other data sources using machine learning and threat intelligence.

Source: What is Amazon GuardDuty?

GuardDuty finding categories:

Category	Examples
Backdoor	EC2 instance communicating with a known command-and-control server
Cryptocurrency	EC2 instance querying cryptocurrency mining pools
Recon	Unusual API calls suggesting reconnaissance activity
Trojan	EC2 instance exhibiting trojan-like behavior
UnauthorizedAccess	API calls from a known malicious IP; root credentials used from an unusual location
Exfiltration	S3 data exfiltration behavior detected
Runtime	Suspicious process execution in ECS, EKS, or EC2 (requires GuardDuty Runtime Monitoring)

GuardDuty protection plans:

- **S3 Protection** – Monitors S3 data events for malicious activity.
- **EKS Protection** – Monitors Kubernetes audit logs and runtime activity.
- **Malware Protection** – Scans EBS volumes attached to EC2 instances for malware.
- **RDS Protection** – Detects anomalous login behavior on RDS databases.
- **Lambda Protection** – Monitors Lambda network activity.
- **Runtime Monitoring** – Agent-based runtime threat detection for EC2, ECS, and EKS.

Remediating GuardDuty findings:

1. Review the finding details — note the affected resource, threat type, and severity.
2. For compromised EC2 instances: isolate the instance (remove from load balancer, apply restrictive security group), capture a forensic snapshot, terminate and replace.
3. For compromised IAM credentials: immediately deactivate the access key or revoke the role's sessions using `aws iam delete-access-key` or an explicit deny policy.
4. For S3 findings: review bucket policies, block public access, enable versioning and MFA delete.

Source: Remediating Runtime Monitoring findings – Amazon GuardDuty

AWS Config

AWS Config continuously records resource configurations and evaluates them against compliance rules. It provides a timeline of configuration changes and can trigger automated remediation.

Enabling AWS Config:

```
# Start the configuration recorder
aws configservice start-configuration-recorder \
  --configuration-recorder-name default

# Put a delivery channel (S3 bucket for config history)
aws configservice put-delivery-channel \
  --delivery-channel '{"name":"default","s3BucketName":"my-config-bucket"}'
```

Config rules and remediation:

1. Create a Config rule (managed or custom Lambda-backed).
2. When a resource is evaluated as **NON_COMPLIANT**, Config can trigger a **remediation action** — an SSM Automation document that fixes the issue.
3. Remediation can be **manual** (triggered on demand) or **automatic** (triggered immediately on non-compliance).

```
# Associate a remediation action with a Config rule
aws configservice put-remediation-configurations \
  --remediation-configurations '[{
    "ConfigRuleName": "s3-bucket-public-read-prohibited",
    "TargetType": "SSM_DOCUMENT",
    "TargetId": "AWS-DisablesS3BucketPublicReadWrite",
    "Automatic": true,
    "MaximumAutomaticAttempts": 3,
    "RetryAttemptSeconds": 60
  }]'
```

Config aggregator collects data from multiple accounts and Regions, enabling organization-wide compliance reporting from a single account.

Amazon Inspector

Amazon Inspector is an automated vulnerability management service that continuously scans EC2 instances, container images in Amazon ECR, Lambda functions, and Lambda layers for software vulnerabilities and unintended network exposure.

Source: Amazon Inspector User Guide

Inspector scanning types:

Scan Type	What It Checks
EC2 scanning	OS and application package vulnerabilities on EC2 instances (via SSM Agent)
ECR container scanning	Vulnerabilities in container images pushed to ECR
Lambda standard scanning	Vulnerabilities in Lambda function code packages and layers
Lambda code scanning	Code vulnerabilities in Lambda function source code

Inspector findings:

- Each finding includes a severity score (based on CVSS), the affected resource, the vulnerability details, and remediation recommendations.
- Findings are published to Security Hub and EventBridge for centralized management and automated response.

Exporting Inspector findings:

```
# Export findings to S3 as a CSV report
aws inspector2 create-findings-report \
  --report-format CSV \
  --s3-destination '{"bucketName":"my-findings-bucket","keyPrefix":"inspector/","kmsKeyArn":"arn:aws:kms:us-east-1:123456789012:key/abc123"}' \
  --filter-criteria '{"severity":[{"comparison":"EQUALS","value":"CRITICAL"}]}'
```

Source: Exporting Amazon Inspector findings reports

Remediating Inspector findings:

1. Review the finding — note the CVE ID, affected package, and fix version.
2. Update the package on the EC2 instance using Systems Manager Patch Manager or Run Command.
3. For container images, rebuild the image with the patched base image and redeploy.
4. For Lambda functions, update the dependency in `requirements.txt` or `package.json` and redeploy.
5. Re-scan to confirm the finding is resolved.

Summary

Domain 4 covers the full spectrum of security and compliance responsibilities for a CloudOps engineer on AWS:

Task	Key Services
IAM features	IAM (password policies, MFA, roles, federation, resource policies, conditions)
Audit and troubleshoot access	CloudTrail, IAM Access Analyzer, IAM Policy Simulator
Multi-account security	AWS Organizations, SCPs, AWS Control Tower, IAM Identity Center
Trusted Advisor remediation	AWS Trusted Advisor, EventBridge, Lambda, Systems Manager Automation
Compliance enforcement	SCPs (Region/service restrictions), AWS Config, conformance packs
Data classification	Resource tagging, Amazon Macie, AWS Config rules
Encryption at rest	AWS KMS (customer managed keys, key policies, rotation, envelope encryption)
Encryption in transit	AWS Certificate Manager (ACM), TLS policies on ALB/CloudFront/API Gateway
Secret storage	AWS Secrets Manager (rotation), Systems Manager Parameter Store
Security findings	AWS Security Hub, Amazon GuardDuty, AWS Config, Amazon Inspector

A strong security posture on AWS combines preventive controls (IAM policies, SCPs, encryption), detective controls (CloudTrail, GuardDuty, Config, Inspector, Security Hub), and responsive controls (automated remediation via EventBridge, Lambda, and Systems Manager Automation).

Sources:

- AWS IAM User Guide
- AWS CloudTrail User Guide
- AWS Organizations User Guide
- AWS KMS Developer Guide
- AWS Certificate Manager User Guide
- AWS Secrets Manager User Guide
- AWS Security Hub User Guide
- Amazon GuardDuty User Guide
- Amazon Inspector User Guide
- AWS Config Developer Guide
- AWS Trusted Advisor
- Data Classification Whitepaper
- Amazon Macie User Guide